Panipat Institute of Engineering and Technology

Samalkha

Computer Science and Engineering (Artificial Intelligence and Machine Learning)

PRACTICAL FILE

"ARTIFICIAL INTELLIGENCE LAB "

BT-CYS-275A

Submitted By:
Submitted To: Name: HARSH TIWARI
Ms. Neha Sharma
Roll No.: 28240572
Asst Professor
Branch: B.Tech. CSE (AI&ML)
CSE(AIML) Dept.
2nd Year (3rd Semester)

Index

S.NO	EXPERIMENT	Date	signature
1	Understand the concept of Artificial Intelligence, explore its types and classify real-world application as AI or non-AI using a python list with proper justification.		
2	Import a dataset using pandas, perform basic statistical analysis by calculating mean, mode , median,variance and standard deviation and interpret the descriptive statistics.		
3	Program to stimulate a simple reflex agent and goal based agent.		
4	Implement Breadth-First search(BFS) and Depth-First search (DFS) in a proclem space.		
5	Stimulate A* search algorithm using heuristic cost function.		
6	Implement Hill-climbing and Genetic Algorithm for optimization problem.		
7	Program to perform resolution and unification in predicate logic.		
8	Develop a program to evaluate a proportional logic statement using truth table.		
9	rRepresents a simple real-world problem using semantic nets or frames.		
10	Stimulate forward and backward chaining in rule-based system.		

Program:-1

AIM : Understand the concept of Artificial Intelligence , explore its types and classify real world applications as AI or Non AI components using a python list with proper justification.

Introduction to AI :- Artificial Intelligence (AI) is the branch of computer science that deals with creating machines or systems capable of performing tasks that normally require **human intelligence**, such as learning, reasoning, problem-solving, decision-making, and understanding natural language.

Definition: "Artificial Intelligence is the science and engineering of making intelligent machines, especially intelligent computer programs."

Applications of AI :-

- **Healthcare:** Disease detection, drug discovery, robotic surgery.
- **Education:** Personalized learning, virtual tutors.
- **Business:** Chatbots, fraud detection, customer insights.
- **Transportation:** Self-driving cars, traffic management.
- **Daily life:** Smart assistants (Alexa, Google Assistant), recommendation systems (Netflix, YouTube).

Components of AI :

1. Machine Learning : Enable system to learn from data

2. Deep Learning : Mimic the human brain to recognize patterns.
3. Natural Language Processing : Helps machines understand and respond to human language
4. Computer Vision : Allows machines to interpret and analyze visual data.
5. Robotics : AI applied in physical machines for performing task .

Types of AI :

1. **Narrow AI (Weak AI):** Performs specific tasks (e.g., voice assistants like Siri).
2. **General AI (Strong AI):** Can perform any intellectual task like a human (still under research).
3. **Super AI:** Hypothetical AI that surpasses human intelligence.

Advantages of AI :

- Increases efficiency and accuracy.
- Handles large volumes of data.
- Reduces human effort in repetitive tasks.
- Supports decision-making with predictive analytics.

Printing the list of AI and Non AI components

Input :-

ai_components = ["Gmail Spam FIletre : Machine Learning", "Self Driving Cars: Deep Learning","Chat Gpt : Natural Language Processing", "Face Id : Computer Vision","Sophia Robot : Robotics"]

non_ai_components = ["Keyboard", "Touchpad", "MIcrophones", "Refrigerators", "Washing machine", "Mouse"]

```
print("===== AI Components =====")

for comp in ai_components:

    print(f"- {comp}")

print("\n===== Non-AI Components =====")

for comp in non_ai_components:

    print(f"- {comp}")
```

Output :

Output

```
===== AI Components =====
- Gmail Spam FIletre : Machine Learning
- Self Driving Cars: Deep Learning
- Chat Gpt : Natural Language Processing
- Face Id : Computer Vision
- Sophia Robot : Robotics

===== Non-AI Components =====
- Keyboard
- Touchpad
- MIcrophones
- Refrigerators
- Washing machine
- Mouse

=== Code Execution Successful ===
```

Program:-2

Aim :- Develop a program to evaluate Proportional Logic Statement using truth table.

Input :-

```
def truth_table():
print("P\tQ\tR\tResult")
print("-" * 25)
Values = [True, False]
for P in values:
for Q in values:
for R in values:
result = (P and Q) or (not R)
print(f"{P}\t{Q}\t{R}\t{result}")
truth_table()
```

Output:-

```
Copyright (
C) 2016 Mic
rosoft Corp
oration. Al
l rights reserved.

PS C:\Users\DIGNITY> & C:/Users/DIGNITY/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/DIG
  File "c:\Users\DIGNITY\Downloads\implement of proportional logic.py", line 9
    result = (P and Q) or (not R)
                                 ^
IndentationError: unindent does not match any outer indentation level
PS C:\Users\DIGNITY> 25
25
PS C:\Users\DIGNITY> & C:/Users/DIGNITY/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/DIG
PS C:\Users\DIGNITY> & C:/Users/DIGNITY/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/DIG
P      Q      R      Result
-------------------------
True   True   True   True
True   True   False  True
True   False  True   False
True   False  False  True
False  True   True   False
False  True   False  True
False  False  True   False
False  False  False  True
PS C:\Users\DIGNITY>
```

Program :- 3

Aim :- Write a program to stimulate a simple reflex agent and goal-based agent.

Input :-

```
# Reflex Agent

def reflex_agent(percept):

    if percept == "Dirty":

        return "clean"

    else:

        return "do nothing"



# Goal-based agent

def goal_based_agent(environment):

    actions = []

    for state in environment:

        if state == "Dirty":

            actions.append("clean")
```

```
        else:
            actions.append("skip")
            return actions


# Test environment
environment = ["Dirty", "Clean"]


# Reflex agent test
print("Reflex agent")
for pos in environment:
    action = reflex_agent(pos)
    print(f"Preceived: {pos}-> Action: {action}")


# Goal_based agent test
print("n\Goal_based agent")
actions = goal_based_agent(environment)
```

```
for i, act in enumerate(actions):
    print(f"position {i}: Action: {act}")
```

Output :-

```
erShell
Copyright (
C) 2016 Mic
rosoft Corp
oration. Al
l rights reserved.

PS C:\Users\DIGNITY\Desktop\c and c++ program> & C:/Users/DIGNITY/AppData/Local/Programs/Python/Python313/python.exe
PS C:\Users\DIGNITY\Desktop\c and c++ program> & C:/Users/DIGNITY/AppData/Local/Programs/Python/Python313/python.exe
PS C:\Users\DIGNITY\Desktop\c and c++ program> & C:/Users/DIGNITY/AppData/Local/Programs/Python/Python313/python.exe
PS C:\Users\DIGNITY\Desktop\c and c++ program> & C:/Users/DIGNITY/AppData/Local/Programs/Python/Python313/python.exe
PS C:\Users\DIGNITY\Desktop\c and c++ program> & C:/Users/DIGNITY/AppData/Local/Programs/Python/Python313/python.exe
p/c and c++ program/# Reflex Agent.py"
c:\Users\DIGNITY\Desktop\c and c++ program\# Reflex Agent.py:28: SyntaxWarning: invalid escape sequence '\G'
  print("n\Goal_based agent")
Reflex agent
Preceived: Dirty-> Action: clean
Preceived: Clean-> Action: do nothing
n\Goal_based agent
position 0: Action: clean
position 1: Action: skip
PS C:\Users\DIGNITY\Desktop\c and c++ program>
```

Program:-4

Aim:- Implement Breadth-First Search (BFS) and Depth-First search (DFS) in a problem space.

Input :-

```
from collections import deque

# Define the graph as an adjacency list

graph = {

    'A': ['B', 'C'],

    'B': ['D', 'E'],

    'C': ['F'],

    'D': [],

    'E': [],

    'F': []

}


# BFS

def bfs(start, goal):

    queue = deque([[start]])
```

```
    visited = set()

    while queue:
        path = queue.popleft()
        node = path[-1]

        if node == goal:
            return path

        if node not in visited:
            visited.add(node)
            for neighbour in graph[node]:
                new_path = list(path)
                new_path.append(neighbour)
                queue.append(new_path)

# DFS
def dfs(start, goal):
    stack = [[start]]
    visited = set()
```

```
    while stack:
        path = stack.pop()
        node = path[-1]

        if node == goal:
            return path

        if node not in visited:
            visited.add(node)
            for neighbour in reversed(graph[node]):
                new_path = list(path)
                new_path.append(neighbour)
                stack.append(new_path)

# Example
start_node = 'A'
goal_node = 'F'

print("BFS path:", bfs(start_node, goal_node))
```

```
print("DFS path:", dfs(start_node, goal_node))
```

Output :-

```
PS C:\Users\DIGNITY\Desktop\c and c++ program> & C:/Users/DIGNITY/AppData/Local/Programs/
p/c and c++ program/# Reflex Agent.py"
BFS path: ['A', 'C', 'F']
DFS path: ['A', 'C', 'F']
PS C:\Users\DIGNITY\Desktop\c and c++ program>
```

Program:-5

Aim:- Write a program to implement Hill Climbing Approach.

Input:-

```
# Implement Hill Climbing Appraoch

print("Name: Kritika, Roll number: 28240595")
import random

def hill_climbing(problem, max_iter=100):
    current = random.choice(problem)  # fixed 'chice' → 'choice'
    for _ in range(max_iter):        # fixed 'for_in' → 'for _ in'
        neighbour = random.choice(problem)  # fixed 'problrem' →
'problem'
        if neighbour > current:
            current = neighbour
    return current                   # moved return outside loop
```

```
problem = [1, 2, 3, 4, 5, 6]

print("Solution:", hill_climbing(problem))
```

Output:-

```
SyntaxError: invalid syntax
PS C:\Users\DIGNITY> & C:/Users/DIGNITY/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/DIGNITY/Downloads/# Implement Hill Climbing A
Name: Kritika, Roll number: 28240595
Solution: 6
PS C:\Users\DIGNITY> & C:/Users/DIGNITY/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/DIGNITY/Downloads/# Implement Hill Climbing A
Name: Kritika, Roll number: 28240595
Solution: 6
PS C:\Users\DIGNITY>
```

Program:-6

Aim:- Write a program to stimulate Forward and Backward Chaining in rule –based systems.

Input :-

```
rules = [
    {"if": ["fever", "cough"], "then": "flu"},
    {"if": ["sore throat"], "then": "cough"},
    {"if": ["flu"], "then": "should rest"}
]
facts = {"fever", "sore throat"}
def forward_chain(facts, rules):
    inferred = set(facts)
    changed = True
    while changed:
        changed = False
        for rule in rules:
            if all(cond in inferred for cond in rule["if"]) and rule["then"] not in inferred:
                inferred.add(rule["then"])   # ← FIXED INDENTATION
                changed = True
    return inferred
print("Forward Chaining Result:", forward_chain(facts, rules))
```

```
# ------------------------------------------------------------
# Forward Chaining (Data-driven)
# ------------------------------------------------------------
def forward_chaining(facts, rules):
    inferred = set(facts)
    changed = True
    while changed:
        changed = False
        for rule in rules:
            if all(cond in inferred for cond in rule["if"]) and rule["then"] not in inferred:
                print(f"Rule fired: IF {rule['if']} THEN {rule['then']}")
                inferred.add(rule["then"])   # ← FIXED INDENTATION
                changed = True
    return inferred


# ------------------------------------------------------------
# Backward Chaining (Goal-driven)
# ------------------------------------------------------------
def backward_chaining(goal, facts, rules, depth=0):
    indent = "  " * depth
    print(f"{indent}Checking goal: {goal}")

    if goal in facts:
        print(f"{indent} Goal '{goal}' is a known fact.")
```

```
        return True

    for rule in rules:
        if rule["then"] == goal:
            print(f"{indent}Trying rule: IF {rule['if']} THEN {goal}")
            all_true = True     # ← FIXED INDENTATION

            for condition in rule["if"]:
                if not backward_chaining(condition, facts, rules, depth + 1):
                    all_true = False     # ← FIXED INDENTATION
                    break

            if all_true:
                print(f"{indent} Goal '{goal}' proven via rule {rule['if']} -> {goal}")
                return True

    print(f"{indent} Goal '{goal}' cannot be proven.")
    return False
# ------------------------------------------------------------
# Main Execution
# ------------------------------------------------------------
print("========== FORWARD CHAINING ==========")
derived_facts = forward_chaining(facts, rules)
print("\nAll derived facts:")
```

```
print(derived_facts)


print("\n========== BACKWARD CHAINING ==========")

goal = "should rest"

result = backward_chaining(goal, facts, rules)

print(f"\nConclusion: '{goal}' is {'TRUE' if result else 'FALSE'}")
```

Output:-

```
PS C:\Users\DIGNITY\Desktop\c and c++ program> & C:/Users/DIGNITY/AppData/Local/Programs/Python/Python313/python.exe
p/c and c++ program/rules = [.py"
Forward Chaining Result: {'should rest', 'flu', 'cough', 'fever', 'sore throat'}
========== FORWARD CHAINING ==========
Rule fired: IF ['sore throat'] THEN cough
Rule fired: IF ['fever', 'cough'] THEN flu
Rule fired: IF ['flu'] THEN should rest

All derived facts:
{'should rest', 'flu', 'cough', 'fever', 'sore throat'}

========== BACKWARD CHAINING ==========
Checking goal: should rest
Trying rule: IF ['flu'] THEN should rest
  Checking goal: flu
  Trying rule: IF ['fever', 'cough'] THEN flu
    Checking goal: fever
     Goal 'fever' is a known fact.
    Checking goal: cough
    Trying rule: IF ['sore throat'] THEN cough
      Checking goal: sore throat
       Goal 'sore throat' is a known fact.
     Goal 'cough' proven via rule ['sore throat'] -> cough
   Goal 'flu' proven via rule ['fever', 'cough'] -> flu
 Goal 'should rest' proven via rule ['flu'] -> should rest

Conclusion: 'should rest' is TRUE
```

Program:-7

Aim:- Develop a program to evaluate a proportional logic statement using truth table.

Input:-

```
import itertools

import re


def evaluate(expression, values):

    expr = expression


    # Replace variables safely using regex

    for var, val in values.items():

        expr = re.sub(rf'\b{var}\b', str(val), expr)


    # Replace symbolic operators with Python equivalents

    expr = expr.replace("!", " not ")

    expr = expr.replace("~", " not ")

    expr = expr.replace("&", " and ")

    expr = expr.replace("|", " or ")


    # Handle implication (A -> B) = (not A or B)

    while "->" in expr:

        expr = re.sub(r'(.+?)->(.+)', r'(not (\1) or (\2))', expr)
```

```
    # Handle biconditional A <-> B
    while "<->" in expr:
        expr = re.sub(r'(.+?)<->(.+)', r'((\1) == (\2))', expr)

    return eval(expr)

# Input
vars_input = input("Enter variables separated by spaces: ")
expression = input("Enter logical expression: ")

variables = vars_input.split()

print("\nTruth Table:")
print(" | ".join(variables) + " | RESULT")
print("-" * 40)

# Generate combinations
for combo in itertools.product([True, False], repeat=len(variables)):
    values = dict(zip(variables, combo))
    result = evaluate(expression, values)

    row = " | ".join(str(v) for v in combo)
    print(f"{row} | {result}")
```

Output:-

Output

Clear

```
Enter variables separated by spaces: p q r
Enter logical expression: (p & q) -> !r

Truth Table:
p | q | r | RESULT
----------------------------------------
True | True | True | False
True | True | False | True
True | False | True | True
True | False | False | True
False | True | True | True
False | True | False | True
False | False | True | True
False | False | False | True

=== Code Execution Successful ===
```

Program:-8

Aim:- Import a dataset using pandas, perform basic statistical analysis by calculating mean, median, mode, variance, and standard deviation and interpret the descriptive statistics.

Input:-

INPUT :-

```
# IMPORTING LIBRARIES

import pandas as pd

# LOAD DATASET

data = pd.read_csv(r"C:\Users\LENOVO\Downloads\Student_Marks.csv")
data
```

OUTPUT:-

	number_courses	time_study	Marks
0	3	4.508	19.202
1	4	0.096	7.734
2	4	3.133	13.811
3	6	7.909	53.018
4	8	7.811	55.299

...	...	...	...
95	6	3.561	19.128
96	3	0.301	5.609
97	4	7.163	41.444
98	7	0.309	12.027
99	3	6.335	32.357

100 rows × 3 columns

```
# HEAD
data.head()
```

OUTPUT:-

number_courses		time_study	Marks
0	3	4.508	19.202
1	4	0.096	7.734
2	4	3.133	13.811
3	6	7.909	53.018
4	8	7.811	55.299

```
#MEAN
```

```
mean_values = data.mean(numeric_only=True)

print("Mean:\n", mean_values)
```

OUTPUT:-

```
Mean:

 number_courses     5.29000

time_study         4.07714

Marks            24.41769

dtype: float64
```

```
# MEDIAN

 median_values = data.median(numeric_only=True)

print("Median:\n", median_values)
```

OUTPUT:-

```
Median:

 number_courses     5.0000

time_study         4.0220

Marks            20.0595

dtype: float64
```

```
#MODE :-

mode_values = data.mode(numeric_only=True)

print("Mode:\n", mode_values)
```

OUTPUT:-

```
Mode:

    number_courses  time_study   Marks

0            3.0      0.096   5.609

1            NaN      0.130   6.053

2            NaN      0.140   6.185

3            NaN      0.156   6.217

4            NaN      0.301   6.349

..           ...        ...     ...

95           NaN      7.711  53.018

96           NaN      7.775  53.158

97           NaN      7.811  53.359

98           NaN      7.909  54.321

99           NaN      7.957  55.299
```

```
# VARIANCE


variance_values = data.var(numeric_only=True)
```

```
print("Variance:\n", variance_values)
```

OUTPUT:-

```
variance:

 number_courses      3.238283

time_study          5.630722

Marks             205.239965

dtype: float64
```

```
#STANDARD DEVIATION

std_values = data.std(numeric_only=True)

print("Standard Deviation:\n", std_values)
```

OUTPUT:-

```
Standard Deviation:

 number_courses     1.799523

time_study         2.372914

Marks             14.326199

dtype: float64
```

Program:-9

Aim:- Build a Minimax algorithm for a two-player game like Tic-Tac-Toe. Extend it with Alpha-Beta pruning.

Input:-

```
import math

# Display the board

def print_board(board):

    for row in board:

        print(row)

    print()

# Check for winner

def check_winner(board):

    # Rows

    for row in board:

        if row[0] == row[1] == row[2] != '-':

            return row[0]

    # Columns

    for c in range(3):

        if board[0][c] == board[1][c] == board[2][c] != '-':

            return board[0][c]

    # Diagonals

    if board[0][0] == board[1][1] == board[2][2] != '-':

        return board[0][0]

    if board[0][2] == board[1][1] == board[2][0] != '-':

        return board[0][2]
```

```
    # Check for empty space → game not over
    for row in board:
        if '-' in row:
            return None
    # Draw
    return "Draw"
# Minimax with Alpha-Beta Pruning
def minimax(board, depth, is_maximizing, alpha, beta):
    result = check_winner(board)


    if result == "X":   # AI wins
        return 10 - depth
    elif result == "O": # Human wins
        return depth - 10
    elif result == "Draw":
        return 0
   if is_maximizing:
        best_score = -math.inf


        for i in range(3):
            for j in range(3):
                if board[i][j] == '-':
                    board[i][j] = "X"     # AI move
                    score = minimax(board, depth + 1, False, alpha, beta)
                    board[i][j] = "-"
```

```
                best_score = max(best_score, score)
                alpha = max(alpha, best_score)
                if beta <= alpha:
                    break
        return best_score
    else:
        best_score = math.inf

        for i in range(3):
            for j in range(3):
                if board[i][j] == '-':
                    board[i][j] = "O"     # Human move
                    score = minimax(board, depth + 1, True, alpha, beta)
                    board[i][j] = "-"
                    best_score = min(best_score, score)
                    beta = min(beta, best_score)
                    if beta <= alpha:
                        break
        return best_score
# Best move for AI using Minimax + Alpha-Beta
def best_move(board):
    best_score = -math.inf
    move = None
    for i in range(3):
        for j in range(3):
```

```
            if board[i][j] == '-':
                board[i][j] = "X"
                score = minimax(board, 0, False, -math.inf, math.inf)
                board[i][j] = "-"

                if score > best_score:
                    best_score = score
                    move = (i, j)
    return move

# ---------------------------------------------------------
board = [
    ['-', '-', '-'],
    ['-', '-', '-'],
    ['-', '-', '-']
]

print("Tic Tac Toe – AI uses X, You use O")
print_board(board)

while True:
    # Human move
    r = int(input("Enter row (0-2): "))
    c = int(input("Enter col (0-2): "))
```

```
    if board[r][c] != "-":
        print("Invalid move! Try again.")
        continue

    board[r][c] = "O"
    print("Board after your move:")
    print_board(board)

    # Check if human won
    if check_winner(board):
        print("Result:", check_winner(board))
        break

    # AI move
    print("AI thinking...")
    ai_r, ai_c = best_move(board)
    board[ai_r][ai_c] = "X"

   print("Board after AI move:")
    print_board(board)
    # Check if AI won
    if check_winner(board):
        print("Result:", check_winner(board))
        break
```

Output:-

```
Tic Tac Toe - AI uses X, You use O
['-', '-', '-']
['-', '-', '-']
['-', '-', '-']

Enter row (0-2): 1
Enter col (0-2): 1
Board after your move:
['-', '-', '-']
['-', 'O', '-']
['-', '-', '-']

AI thinking...
Board after AI move:
['X', '-', '-']
['-', 'O', '-']
['-', '-', '-']
```

Output

Clear

```
Enter row (0-2): 2
Enter col (0-2): 2
Board after your move:
['X', '-', '-']
['-', 'O', '-']
['-', '-', 'O']

AI thinking...
Board after AI move:
['X', '-', 'X']
['-', 'O', '-']
['-', '-', 'O']
```

```
Enter row (0-2): 1
Enter col (0-2): 2
Board after your move:
['X', '-', 'X']
['-', 'O', 'O']
['-', '-', 'O']

AI thinking...
Board after AI move:
['X', 'X', 'X']
['-', 'O', 'O']
['-', '-', 'O']

Result: X

=== Code Execution Successful ===
```

Program:-10

Aim:- Write a program to perform resolution and unification in predicate logic.

Input:-

```
# -------- UNIFICATION ALGORITHM --------


def is_variable(x):
    return isinstance(x, str) and x[0].islower()  # variables start with lowercase




def unify(x, y, substitution):
    if substitution is None:
        return None
    elif x == y:
        return substitution
    elif is_variable(x):
        return unify_var(x, y, substitution)
    elif is_variable(y):
        return unify_var(y, x, substitution)
    elif isinstance(x, list) and isinstance(y, list):
        if len(x) != len(y):
            return None
        for a, b in zip(x, y):
            substitution = unify(a, b, substitution)
        return substitution
```

```
    else:
        return None



def unify_var(var, x, substitution):
    if var in substitution:
        return unify(substitution[var], x, substitution)
    elif x in substitution:
        return unify(var, substitution[x], substitution)
    else:
        substitution[var] = x
        return substitution



# -------- RESOLUTION ALGORITHM --------


def resolve(clause1, clause2):
    new_clause = []

    for literal1 in clause1:
        for literal2 in clause2:
            if literal1[0] == '¬' and literal1[1] == literal2[0]:
                # unify the arguments
                args1 = literal1[2:-1].split(',')
                args2 = literal2[2:-1].split(',')
```

```
            substitution = unify(args1, args2, {})

            if substitution is not None:
                # remove complementary literals
                c1 = [l for l in clause1 if l != literal1]
                c2 = [l for l in clause2 if l != literal2]

                # apply substitution
                c1 = substitute_clause(c1, substitution)
                c2 = substitute_clause(c2, substitution)

                new_clause = c1 + c2
                return new_clause

    return None



def substitute_clause(clause, substitution):
    new_clause = []
    for literal in clause:
        pred = literal[0]
        args = literal[2:-1].split(',')
        new_args = []

        for a in args:
```

```
            if a in substitution:
                new_args.append(substitution[a])
            else:
                new_args.append(a)

        new_literal = pred + "(" + ",".join(new_args) + ")"
        new_clause.append(new_literal)

    return new_clause



# -------- MAIN PROGRAM --------


def main():
    print("\n=== Predicate Logic Unification and Resolution ===")

    clause1 = ["¬P(x)", "Q(x)"]
    clause2 = ["P(a)"]

    print("\nClause 1:", clause1)
    print("Clause 2:", clause2)

    result = resolve(clause1, clause2)

    if result is None:
```

```
        print("\nResolution failed — clauses cannot be resolved.")
    else:
        print("\nResolved Clause:", result)


if __name__ == "__main__":
    main()
```

Output:-

Output Clear

```
=== Predicate Logic Unification and Resolution ===

Clause 1: ['¬P(x)', 'Q(x)']
Clause 2: ['P(a)']

Resolved Clause: ['Q(x)']

=== Code Execution Successful ===
```